

KJO Liquid Rocket Motor Control System

System Architecture and Operational Overview

Ken Overton

KJO Liquid Rocket Motor Control System

Version: 0.2

Date: September 2026

This document provides a high-level structural and functional overview of the KJO control system for a bi-propellant liquid rocket motor. It describes the four electronic subsystems (RCU, EMU, GSEMU, LCMU), the propellant flow architecture, the inter-unit communication protocols, and the complete command set. Detailed subsystem reference documents are listed in [Section 6](#).

Contents

1	Introduction	2
2	Propellant System	2
2.1	Flow Architecture	2
2.2	Fill Sequence	3
2.3	Engine Firing Sequence	4
2.4	Dump / Safe Procedure	4
2.5	Pressurization	5
3	Electronic Control Architecture	5
3.1	RCU-EMU Radio Link	5
3.2	EMU-GSEMU-LCMU CAN Bus	5
4	Shared Libraries	6
5	Command Set	6
5.1	Packet Structures	6
5.2	Command Table	6
6	Document References	8
7	Version History	9

1 Introduction

The KJO system controls a bi-propellant liquid rocket motor using nitrous oxide (N_2O) as the oxidizer and a liquid fuel with regenerative cooling. Four autonomous microcontroller units share control responsibilities:

- **Remote Control Unit (RCU)** — handheld operator terminal. Sends commands over a 900 MHz LoRa radio link; displays system status on a 3.5" colour touchscreen.
- **Engine Management Unit (EMU)** — mounted in the rocket. Receives commands from the RCU, actuates the Oxidizer Feed and Fuel Feed servos, reads all sensors, and coordinates the Fill valve and load-cell readings with the GSEMU and LCMU over a shared CAN bus.
- **GSE Management Unit (GSEMU)** — mounted on the launch pad. Controls the Fill valve servo and the umbilical quick-release servo; coordinates the fill-to-target sequence with the LCMU and reports status to the EMU, both over the shared CAN bus. A separate, ground-referenced disconnect-sense line (not CAN) gives both EMU and GSEMU instant, protocol-independent detection of umbilical separation.
- **Load Cell Management Unit (LCMU)** — mounted on the test stand. Measures Ox load during oxidizer fill (supporting GSEMU's fill-to-target sequence) and thrust during test-stand burns, logging both to microSD; answers CAN queries from EMU and GSEMU.

RCU, EMU, and GSEMU are based on the Adafruit Feather M0 (ATSAMD21G18A, 48 MHz ARM Cortex-M0+, 3.3 V logic); LCMU is based on the Adafruit Feather RP2040 Adalogger (dual-core Cortex-M0+, 133 MHz). All four run firmware written in C++17 using the Arduino / PlatformIO framework.

2 Propellant System

2.1 Flow Architecture

Figure 1 shows the propellant system schematic.

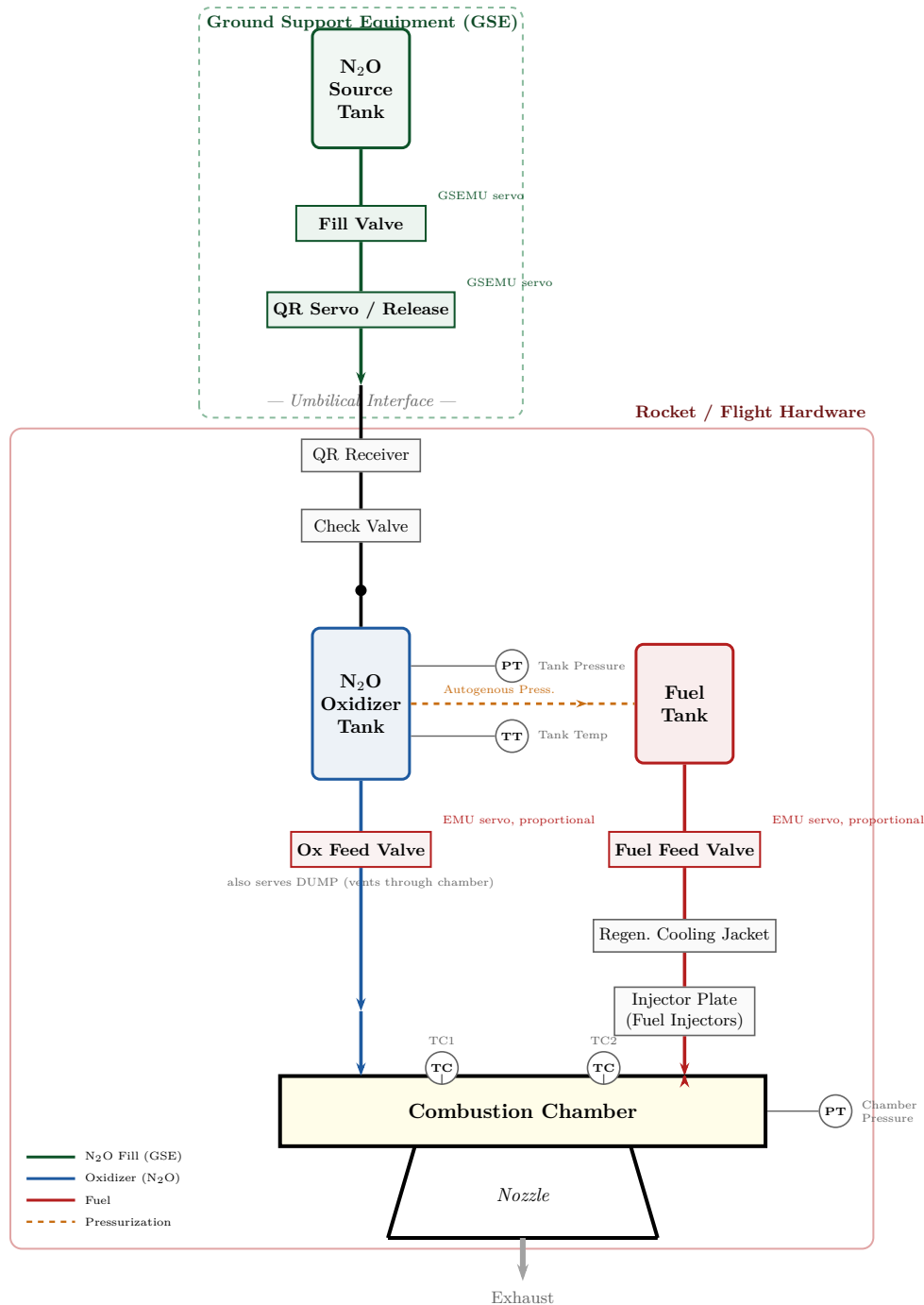


Figure 1: Propellant system schematic. Green lines: N₂O fill path (GSE). Blue lines: oxidizer feed path. Red lines: fuel feed path. Orange dashed: autogenous pressurization.

2.2 Fill Sequence

EMU owns the entire fill session — GSEMU has no fill state machine of its own, it only opens/closes the Fill valve servo on command; LCMU only answers weight queries. This replaced an earlier design where GSEMU tracked progress itself (see the CAN protocol note in Section 3).

1. Operator commands **BEGIN_FILL** from the RCU.
2. EMU (fresh start only — a paused session skips this) sends **CAN_TARE** to LCMU.
3. EMU sends **CAN_OPEN_FILL_VALVE** to GSEMU.
4. N₂O flows: GSE source tank → Fill valve → QR servo → umbilical → check valve → on-board oxidizer tank.
5. EMU polls LCMU with **CAN_REPORT_CURRENT_WEIGHT** every 200 ms, comparing the tare-relative reading against a per-motor target it holds locally (never sent to GSEMU or LCMU).
6. Once the target is reached, EMU sends **CAN_CLOSE_FILL_VALVE** to GSEMU. **STOP_FILL** (operator pause) does the same at any point; a later **BEGIN_FILL** resumes without re-taring.
7. The RCU polls fill progress locally from the EMU via **QUERY_FILL_PROGRESS** (LoRa) — this is answered from EMU's own cached poll state, not a fresh CAN round trip.

2.3 Engine Firing Sequence

Ignition is a 10-step sequence (**Check_Engine_Start()**) that establishes propellant flow *before* firing the igniter — reversed from an earlier design that fired first. Fuel opens before Ox, since it has the longer physical path to the chamber and needs the head start; the igniter's solid starter motor only burns for a very short window once lit.

1. Operator selects a source with **SET_IGNITION_SOURCE** (RCU / LCO / Airstart) and arms it with **ARM_LAUNCH** (renamed from **ENABLE_LCO_CONTROL**).
2. Operator commands **START_ENGINE** (RCU source), or the armed LCO/Airstart trigger fires — either way, EMU sends **ENGINE_START_ARMED** to the RCU unprompted the instant the sequence begins.
3. EMU verifies Ox and Fuel are closed; if GSEMU is CAN-reachable, closes the Fill valve and waits for confirmation (protects the ground tanks from an open Fill valve during ignition).
4. EMU releases the umbilical quick-release and waits for confirmed physical separation, then a fixed dwell.
5. EMU opens the Fuel valve, waits for confirmed open plus a flow delay, then opens the Ox valve the same way.
6. EMU fires the igniter relay — the last step, once both propellants are already flowing.
7. Proportional control commands (**OXIDIZER_FLOW_PROP**, **FUEL_FLOW_PROP**) may be sent during burn to adjust mixture ratio.
8. **STOP_ENGINE_FLOWS** closes both feed valves to terminate thrust.

Any precondition or confirmation failure at steps 3–5 aborts the sequence; see the EMU Reference for the full step table and failure codes.

2.4 Dump / Safe Procedure

The **DUMP** command opens the Oxidizer Feed valve (EMU-controlled), venting the on-board N₂O tank pressurant through the combustion chamber. There is no dedicated Dump valve. The RCU Dump button has latching behaviour — a second press sends **STOP_DUMP** and closes the Oxidizer Feed valve.

2.5 Pressurization

The on-board N_2O tank is self-pressurizing (autogenous). The vapour-space pressure is communicated to the fuel tank via a dedicated pressurization line, eliminating the need for a separate pressurant.

3 Electronic Control Architecture

Figure 2 shows the four-unit electronic control architecture.

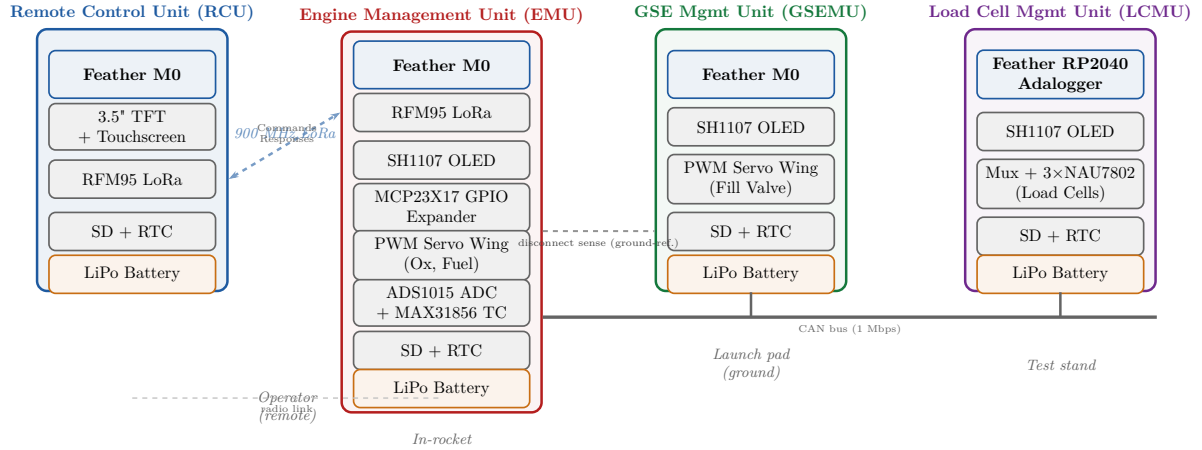


Figure 2: Electronic control architecture. The RCU communicates with the EMU over a 900 MHz LoRa radio link. EMU, GSEMU, and LCMU share a CAN bus (1 Mbps); a separate ground-referenced line gives EMU and GSEMU instant umbilical disconnect-sense, independent of CAN.

3.1 RCU–EMU Radio Link

The link uses the HopeRF RFM95W module (Semtech SX1276 chipset) operating at 915.0 MHz with 23 dBm transmit power. Packets are simple binary structures (see Section ??). The RCU sends a **HEARTBEAT** packet at 5-second intervals when no other command is pending; the EMU echoes a **HEARTBEAT** response containing valve positions and battery voltage. Link loss is declared after 30 seconds without a response.

3.2 EMU–GSEMU–LCMU CAN Bus

EMU, GSEMU, and LCMU share one CAN bus (MCP2515 controllers, 1 Mbps, node IDs EMU = 1, GSEMU = 2, LCMU = 3) — addressed command/response only, no unsolicited broadcasts. This replaced an earlier wired GPIO CMD/STATUS handshake that carried Fill-valve coordination directly between EMU and GSEMU; that handshake's two lines (**AUX_IO_1/AUX_IO_2** on EMU's MCP23X17 GPIO expander) are retired. Full command tables and protocol details for each unit are in their own reference documents (Section 6).

Separately, a ground-referenced disconnect-sense line on each side of the EMU–GSEMU umbilical (not CAN) gives both units instant physical-disconnect detection independent of bus traffic or timeout logic.

4 Shared Libraries

Common definitions reside in `KJO_Shared_Libraries` and are shared across all four firmware projects via PlatformIO library dependencies:

Library	Contents	Used by
<code>KJO_System_Config</code>	Platform IDs, version strings	RCU, EMU, GSEMU, LCMU
<code>KJO_Command_Defs</code>	LoRa command codes, packet structs	RCU, EMU
<code>KJO_CAN_Command_Defs</code>	CAN command codes, packet structs, node IDs	EMU, GSEMU, LCMU
<code>KJO_Radio</code>	Radio pin assignments, frequency	RCU, EMU
<code>KJO_RCU_Display</code>	TFT GUI functions and data types	RCU

5 Command Set

5.1 Packet Structures

Six binary packet types are used on the radio link (all defined in `KJO_Command_Defs.h`):

- **Command_Payload** (4 bytes, RCU→EMU): `command` (1B), `param` (short, 2B), `terminator` (1B). Used for most commands.
- **Command_Return_Packet** (4 bytes, EMU→RCU): `command` (1B), `status` (bool, 1B), `r_val` (short, 2B). Returned for most non-heartbeat commands.
- **Heartbeat_Command_Packet** (1 byte, RCU→EMU): `command` only — the RCU already knows its own battery voltage locally.
- **Heartbeat_Response_Packet** (11 bytes, EMU→RCU): `command` (1B), `emu_voltage` (float, 4B), `fill_pct`, `ox_pct`, `fuel_pct` (int8, 1B each), `qr_connected`, `gsemu_reachable`, `lcmu_reachable` (bool, 1B each). The last three come from EMU's own CAN presence cache, not a fresh query per heartbeat.
- **Fill_Progress_Response_Packet** (9 bytes, EMU→RCU, answers `QUERY_FILL_PROGRESS`): `command` (1B), `fill_state` (1B: `IDLE`/`IN_PROGRESS`/`PAUSED`/`COMPLETE`/`ABORTED`), `current_weight_lbm` (float, 4B), `percent` (int8, 1B), `seconds_remaining` (int16, 2B; -1 until enough flow has been observed to estimate).
- **RCU_Status_Response_Packet** (9 bytes, EMU→RCU, answers `RCU_STATUS`): `command` (1B), `emu_health`, `gsemu_health`, `lcmu_health` (uint16, 2B each, from live `CAN_GET_HEALTH_STATUS` queries), `gsemu_reachable`, `lcmu_reachable` (bool, 1B each).

A seventh type, **Extended_Heartbeat_Response_Packet**, exists only for `RCU_UNIT_TEST`'s bench-test telemetry panel and is never sent to or parsed by production RCU.

5.2 Command Table

Codes 1–35 below are production RCU's full command set. Codes 36–39 (`FIRE_IGNITER`, `OPEN_FILL_VALVE`, `CLOSE_FILL_VALVE`, `EXTENDED_HEARTBEAT`) exist only for `RCU_UNIT_TEST`, a separate bench-test harness firmware with an expanded command set for exercising individual actuators directly; production RCU never sends them and this

table omits their detail.

Code	Name	Direction	param	Description
1	BEGIN_FILL	RCU→EMU	—	Start/resume the fill session (EMU-owned; see Fill Sequence)
2	STOP_FILL	RCU→EMU	—	Pause the active fill session
3	OPEN_OXIDIZER_FEED	RCU→EMU	—	Open Ox Feed valve fully
4	CLOSE_OXIDIZER_FEED	RCU→EMU	—	Close Ox Feed valve
5	OPEN_FUEL_FEED	RCU→EMU	—	Open Fuel Feed valve fully
6	CLOSE_FUEL_FEED	RCU→EMU	—	Close Fuel Feed valve
7	OXIDIZER_FLOW_PROP	RCU→EMU	%	Set Ox valve position (0–100%)
8	FUEL_FLOW_PROP	RCU→EMU	%	Set Fuel valve position (0–100%)
9	DUMP	RCU→EMU	—	Open Ox valve; vent pressurant through chamber (no dedicated Dump valve)
10	START_ENGINE_FLOWS	RCU→EMU	—	Open both feed valves simultaneously (bench/test use, not the ignition sequence)
11	STOP_ENGINE_FLOWS	RCU→EMU	—	Close both feed valves simultaneously
12	ARM_LAUNCH	RCU→EMU	0/1	Arm/disarm the currently-selected ignition source (renamed from ENABLE_LCO_CONTROL); rejected while a fill is armed
13	START_ENGINE	RCU→EMU	—	Begin the 10-step ignition sequence (RCU source)
14	RESET	RCU→EMU, or EMU→RCU unprompted	—	Abort: sequenced vent procedure ending in a terminal latched state (power-cycle only recovery); unprompted with r_val = 1 means EMU auto-aborted on sustained Ox tank overpressure
15	READ_TANK_PRESSURE	RCU→EMU	—	Return oxidizer tank pressure
16	READ_CHAMBER_PRESSURE	RCU→EMU	—	Return chamber pressure
17	READ_TEMP2	RCU→EMU	—	Return EMU's Temp2 thermocouple reading (renamed from READ_TANK_TEMP ; physical mounting location not fixed)
18	READ_TEMP1	RCU→EMU	—	Return EMU's Temp1 thermocouple reading (renamed from READ_CHAMBER_TEMP)
19	START_PRESSURE_RECORDING	RCU→EMU	—	Begin high-rate pressure logging
20	STOP_PRESSURE_RECORDING	RCU→EMU	—	End high-rate pressure logging
21	READ_VALVE_POSITION	RCU→EMU	valve#	Return servo position (0–100%)
22	HEARTBEAT	Bidirectional	—	Keepalive / link maintenance, 5 s interval
23	RCU_STATUS	RCU→EMU	—	Request EMU/GSEMU/LCMU health summary (live CAN queries)
24	STOP_DUMP	RCU→EMU	—	Close Ox valve (second Dump-button press)
25	QR_RELEASE	RCU→EMU	—	Command GSEMU to release the umbilical quick-release
26	QUERY_FILL_PROGRESS	RCU→EMU	—	Poll fill session state/weight/percent/time-remaining (renamed from QUERY_FILL_STATUS); answered from EMU's own cache

Code	Name	Direction	param	Description
27	TARE_LCMU	RCU→EMU (CAN relay)	—	Relay: command LCMU to tare
28	GET_LCMU_BATTERY	RCU→EMU (CAN relay)	—	Relay: request LCMU battery voltage
29	REPORT_LCMU_WEIGHT	RCU→EMU (CAN relay)	—	Relay: request LCMU's current weight/thrust reading
30	BEGIN_WEIGHT_RECORDING	RCU→EMU (CAN relay)	—	Relay: start LCMU weighing-mode SD logging
31	STOP_WEIGHT_RECORDING	RCU→EMU (CAN relay)	—	Relay: stop LCMU weighing-mode SD logging
32	BEGIN_THRUST_RECORDING	RCU→EMU (CAN relay)	—	Relay: arm LCMU thrust-mode capture
33	STOP_THRUST_RECORDING	RCU→EMU (CAN relay)	—	Relay: manual override to stop LCMU thrust-mode capture
34	SET_IGNITION_SOURCE	RCU→EMU	0/1/2	Select ignition source: 0=RCU, 1=LCO, 2=Airstart; rejected while armed
35	ENGINE_START_ARMED	EMU→RCU, unprompted	—	Sent the instant any source arms the 10-step sequence; <code>r_val</code> = ignition source (same encoding as <code>SET_IGNITION_SOURCE</code>)

6 Document References

Document	File	Contents
RCU Reference	RCU/KJO_RCU_Reference.pdf	Hardware, software, GUI
EMU Reference	EMU/KJO_EMU_Reference.pdf	Hardware, software, valve control
GSEMU Reference	GSEMU/KJO_GSEMU_Reference.pdf	Hardware, software, Fill valve
LCMU Reference	LCMU/KJO_LCMU_Reference.pdf	Hardware, software, CAN bus, load cells

7 Version History

Version	Date	Notes
0.1	March 2026	Initial release.
0.2	September 2026	Added LCMU as a fourth unit throughout (Introduction, Electronic Control Architecture and its figure, Shared Libraries, Document References). Replaced the retired EMU–GSEMU wired GPIO handshake description with the current EMU–GSEMU–LCMU CAN bus. Rewrote Fill Sequence for the current EMU-owned (not GSEMU-owned) fill session. Rewrote Engine Firing Sequence for the current 10-step, propellant-flow-before-ignition, Fuel-before-Ox sequence (was: simultaneous valves, igniter first). Fully refreshed Packet Structures (added Fill_Progress_Response_Packet and RCU_Status_Response_Packet; corrected Heartbeat_Response_Packet’s fields) and the Command Table (24 stale entries replaced with the current 35-command production set; several codes had been reassigned since the last edit, not just added to).
